

Team Yukti

Technical Documentation

Copi, a parenting co-pilot, and the polypharmacy risk detection engine

1. Overview

This repository contains two products that share one codebase, one Express process, one PostgreSQL database and one session cookie.

1. A polypharmacy risk detection engine. A Graph Neural Network trained on a biological drug-protein-protein interaction graph that scores drug combinations against 1,317 known polypharmacy side effects, exposed through a FastAPI service and a clinician-facing React analyzer.
2. Copi, a co-pilot for new parents. Copi joins a family group chat, passively builds a structured health record of a child from ordinary conversation, answers when it is addressed, checks every new prescription against what the child is already taking by calling the interaction engine above, and comes back later to check in.

Copi is the larger and more novel of the two systems, and the majority of this document is about it. The interaction engine is documented in enough depth to explain the contract Copi depends on and how the model itself works.

1.1 Technology stack

Layer	Technology
Machine learning model	PyTorch and PyTorch Geometric, GraphSAGE encoder
Model service	FastAPI, Uvicorn, Python 3.10
Application server	Node.js 22, Express 4, CommonJS
ORM and database	Sequelize 6, PostgreSQL 16
Job queue	Custom PostgreSQL backed queue with SKIP LOCKED
Runtime validation	Zod 4 at every boundary
Language model	Gemini 3.5 Flash Lite, with Ollama and fixture drivers
Frontend	React 19, Vite, React Router 7, Tailwind CSS 4, Radix UI
Chat channels	Telegram long polling, WhatsApp Cloud API webhook, direct upload
Notifications	Web Push over VAPID
PDF export	jsPDF on the client, hand written PDF 1.4 writer on the server

2. Copi

Copi is a subsystem inside the existing backend, mounted at `/api/v1`, with every database table prefixed `copi_`. It is strictly additive. No existing route, service, model or component was modified beyond mounting it, and setting `COPI_ENABLED=false` removes it entirely and leaves the analyzer exactly as it was.

2.1 The problem it solves

A child's health history in most families already exists, scattered across a group chat. Photographs of rashes, a prescription slip someone photographed at the pharmacy, a message saying the fever started on Tuesday, a reply three days later saying it cleared up. None of it is structured, none of it is searchable, and none of it is available at the moment a doctor asks how long this has been going on.

Copi sits in that chat and turns it into a record without anyone changing how they talk. It extracts events, groups them into episodes, maintains a structured profile of the child, checks new medicines against the active list, and answers questions with the child's own history in context.

2.2 Directory structure

```
backend/src/copi/  
  config.js    layered over the existing environment config  
  index.js     mount point, boot sequence, seeding  
  models/      29 Sequelize models grouped by domain  
  lib/         uuidv7, age maths, language detection, EXIF, vectors, tokens  
  llm/         provider, drivers, schemas and ten committed prompt files  
  modules/     safety, twin, guidance, medications, ddi, growth, media,  
               episodes, events, followups, vaccinations, notifications, export  
  ingestion/   the twelve stage pipeline and the channel adapters  
  jobs/        the PostgreSQL backed queue and 16 handlers  
  routes/      everything served under /api/v1  
  data/        WHO growth tables, Nepal EPI schedule, drug aliases, demo seed
```

2.3 The ingestion pipeline

Every inbound message travels through twelve explicit, individually resumable stages. Each stage records its progress on the message row, so a failure is diagnosable as a named stage and retryable from where it stopped. Stage 1 runs synchronously in the webhook or poll handler so the channel gets an immediate acknowledgement. Stages 2 through 12 run as a background job.

Stage	Name	What it does
1	receive	Persists the raw message. A unique constraint on (<code>threadId</code> , <code>externalMessageId</code>) makes a replayed webhook or an overlapping long poll window a no-op.

2	resolve_thread	Maps the chat to a child. An unbound thread gets a six digit link code once and then nothing is ingested from it.
3	resolve_sender	Maps the sender to a Copi account. An unlinked sender is recorded and still builds the timeline, but can never make Copi answer questions about the child.
4	mention_gate	Decides whether Copi was addressed. An @mention, the bare bot name at the start, or a reply to one of Copi's own messages. In a one to one chat the gate is skipped, since there is nobody else to be talking to.
5	detect_language	Classifies the text as en, ne, ne-Latn or mixed.
6	media_fetch	Downloads and normalizes attachments, strips EXIF, writes media rows.
7	media_classify	Classifies each attachment as a skin photo, a prescription, a lab report, a growth chart or other.
8	media_analyze	Runs the matching deep analysis prompt for that class.
9	safety	The deterministic red flag engine plus a separate model triage call. Binding, and evaluated before any guidance is generated.
10	extract_events	Persists extracted candidate events, links them to episodes, and links the message to any check-in it answers.
11	twin_update	Enqueues embedding indexing for confirmed events and debounces a twin rebuild.
12	respond	Chooses the response shape, generates the answer, sends it, and schedules a follow-up check-in when something is being watched.

One ordering detail is deliberate and is worth stating plainly. The extraction call runs before the safety evaluation, but nothing is persisted until after safety has run. The deterministic matcher scans extracted symptoms and structured numbers as well as raw text, and a value such as temperature_c: 38.1 only exists after extraction. The guarantee that matters is preserved exactly: safety is evaluated before any guidance is generated, and its result is binding.

Persistence of extracted events is individually guarded. A malformed candidate costs the timeline entry, never the conversation and never the reply.

2.4 The safety layer

Copi answers questions about infants. The safety layer is therefore deterministic code, not a prompt instruction. A prompt is a request; this has to be a guarantee. The implementation lives in `backend/src/copi/modules/safety/` and is covered by 48 test cases across `safety.test.js` and `tone.test.js`.

2.4.1 Tiers and precedence

Tier	Meaning	Behaviour
emergency	Needs care right now	Leads with the action. Gives the Nepal ambulance number and the configured hospital. No home remedies at all. The event is still logged.
urgent	Should be seen today or within 24 hours	Says so clearly and early, then interim comfort measures.
monitor	Reasonable to manage at home while watching	Home care steps, an explicit watch list, and an automatic check-in.
info	A question or a log entry	Answered normally.

The precedence rule is a single line:

$$\text{finalTier} = \max(\text{ruleTier}, \text{llmTier})$$

The rule engine produces `ruleTier` from seeded, age gated rules. The model is asked separately for `llmTier`, through a prompt that produces no parent facing text at all. The model may escalate. It may never de-escalate. Every evaluation, including every disagreement between the two, is written to `copi_safety_evaluations` and exposed at `GET /api/v1/children/:childId/safety-evaluations`. That table is the audit trail.

At emergency tier the response is assembled from the rule row's own action text. The model contributes exactly one sentence acknowledging what the parent described. Everything else is fixed.

2.4.2 The red flag rules

Nineteen rules are seeded into `copi_red_flag_rules` and are editable there. Age gates are expressed in days, inclusive of `minAgeDays` and exclusive of `maxAgeDays`.

Code	Age window	Tier	Fires on
NEONATAL_FEVER	under 90d	emergency	temperature at or above 38.0 C
FEVER_PROLONGED	any	urgent	fever reported for 72 hours or more
FEVER_HIGH_INFANT	90d to 2y	urgent	temperature at or above 39.0 C
BREATHING_DIFFICULTY	any	emergency	retractions, grunting, nasal flaring, fast

			breathing, wheeze with distress
CYANOSIS	any	emergency	blue or grey lips, tongue or face
SEIZURE	any	emergency	convulsion, fit, stiffening with unresponsiveness
UNRESPONSIVE_LETHARGY	any	emergency	floppy, will not wake, not responding
NON_BLANCHING_RASH	any	emergency	does not fade under pressure, petechiae, purpura
DEHYDRATION	any	urgent, emergency under 90d	no wet nappy for 8 hours, sunken fontanelle, no tears, sunken eyes
BILIOUS_VOMIT	any	emergency	green vomit, projectile vomiting
BLOOD_IN_STOOL_OR_VOMIT	any	urgent	blood in stool or vomit
HEAD_INJURY_SEVERE	any	emergency	loss of consciousness, vomiting after a head injury, unequal pupils
POISONING	any	emergency	swallowed medicine, chemical or a battery
BURN_SIGNIFICANT	any	urgent	blistering, larger than the child's palm, face, hands or genitals
FEEDING_REFUSAL_NEONATE	under 90d	urgent	refusing feeds
STIFF_NECK_PHOTOPHOBIA	any	emergency	stiff neck, discomfort with light
SEVERE_ALLERGIC	any	emergency	facial, lip or tongue swelling
PROLONGED_JAUNDICE	under 180d	urgent	yellowing beyond two weeks, or spreading
DDI_MAJOR	any	urgent	the interaction model returns major or contraindicated

2.4.3 Matching

The matcher takes three inputs: the raw message text in either language, the symptom extraction lifted verbatim, and structured numeric fields.

- Keyword matching uses word boundaries for single Latin words, so fit matches he had a fit and not fitting the nappy. Devanagari and multi word phrases use substring matching, where boundaries do not apply the same way.
- Matching is bilingual throughout. Devanagari and Romanized Nepali forms are both present for fever, difficulty breathing, convulsion, vomiting and blood.

- Rules marked requiresAll need every configured matcher to fire. FEVER_PROLONGED needs both a duration of at least 72 hours and a fever symptom, so a three day cough does not fire it.
- escalateBelowAgeDays lives in code rather than in the rules table. Dehydration is urgent in a toddler and an emergency in a newborn, which is behaviour rather than editable content.

The age boundary is tested explicitly in both directions:

38.1 C at 88 days -> emergency (NEONATAL_FEVER)

38.1 C at 90 days -> not emergency

38.1 C at 400 days -> monitor

2.4.4 Tone enforcement

Tone is enforced twice: once in the prompt, and again in a post generation validator, toneLinter.js, that runs on every answer before it is sent. The target is concerned, not frightening. A parent reading this at two in the morning should feel steadier afterwards.

Rejected outright:

- The words emergency as an exclamation, dangerous, life-threatening, serious condition, you must
- All capital words, with an allow list for acronyms such as ORS, WHO and BCG
- More than one exclamation mark anywhere
- A stated diagnosis. Your child has X is rejected; this often looks like X is fine
- Prescription drug dosing in any form
- Over the counter dosing without an instruction to confirm with a pharmacist
- Recommending anything on the child's recorded allergy list
- Home remedies at emergency tier

The required shape for monitor and urgent answers:

3. One sentence acknowledging what the parent noticed
4. What it commonly is, hedged
5. Two to four concrete home care steps
6. An explicit check with a doctor if you notice line
7. A promise to check in tomorrow

On failure the answer is regenerated once with the specific violations fed back. If that also fails, a fixed template is sent instead, and the template is itself checked against the linter in the test suite, so the fallback can never violate the spec it exists to protect. The medical disclaimer appears once per episode rather than once per message, because nobody reads the fifth disclaimer.

2.4.5 Response shaping

Not every message deserves a five part clinical answer. responseMode.js decides the shape first, deterministically, and only then picks a prompt.

Mode	Shape	Chosen when
------	-------	-------------

full	Acknowledgement, hedged framing, home care, watch list, check-in promise	The binding tier is monitor, urgent or emergency, or an ongoing symptom was extracted, or a symptom was written to the record
brief	A couple of sentences answering what was asked, no watch list and no promise	A question at info tier with nothing being watched
confirm	One line confirming what was written to the record	A pure logging message, and the parent's only feedback that their message updated anything

The rule that keeps this safe is that the safety tier is binding and can only push the response up. Nothing in the shaping logic can turn an urgent message into an acknowledgement.

2.5 Multi-tenancy and access control

`assertChildAccess(userId, childId, minPermission)` is the only path to a child's data. Every child scoped route and every retrieval query goes through it, and `child_id` is filtered in the database WHERE clause, never after the fact. Multi-tenant leakage is the worst possible bug in this product, and that guarantee does not depend on the storage format of anything.

Role	Can read	Can write events, media, medicines	Can manage caregivers	Can delete the child
owner	yes	yes	yes	yes
editor	yes	yes	no	no
viewer	yes	no	no	no

In a group chat, a sender who is not linked to a Copi account is recorded and their messages still build the timeline, but they cannot make Copi answer anything about the child. A stranger in a family group must not be able to query a child's medical history. The access control matrix is exercised by 69 integration probes in `tests/rbac.integration.js`.

2.6 The digital twin

The twin is the structured, current picture of the child that goes into every guidance prompt. It is rebuilt on a debounce, coalescing a burst of messages into a single rebuild.

The rebuild is deliberately split. Structured facts are derived deterministically from the record: active medications, the newest reading of each growth field, open episodes, allergies. Only the narrative summary is generated. An allergy is not something to re-infer from prose on every rebuild. It is a row, carried forward, and the model is allowed to add to the list but never to quietly drop from it.

Growth readings are collected from several recent rows rather than one, because weight and height are recorded separately as often as together. Taking only the newest row would report height not recorded for a child measured last week purely because today's entry was a weigh-in.

2.7 Retrieval and the ask feature

Guidance answers are grounded in the child's own history through embedding retrieval. Events, episodes, profile text, media analyses and messages are chunked at roughly 900 characters, embedded at 768 dimensions, and stored in `copi_embeddings`. Indexing is idempotent on (`sourceType`, `sourceId`, `chunkIndex`), so an at-least-once job re-run replaces rather than duplicates, and leftover chunks from a longer previous version of a source are removed.

Retrieval takes the top 8 chunks within a cosine distance ceiling of 0.55. Chunks further away are dropped rather than padded into the prompt. The whole context is assembled against a 6,000 token budget.

Every answer is explainable. The why did it say that view, served by `GET /api/v1/guidance/:guidanceId/context`, shows the exact events that were in context, the retrieved passages with their cosine distances, the token budget used, and the reconciliation between the deterministic rule engine and the model's own tier suggestion.

2.8 Media handling

- Attachments are size limited at 25 MB and restricted to JPEG, PNG, WebP, HEIC, HEIF, PDF and MP4.
- EXIF handling extracts `DateTimeOriginal` into a first class `captured_at` column, then removes the entire APP1 and APP2 segment before the bytes are stored. Surgically deleting only the GPS IFD would mean rewriting the TIFF tree and fixing every offset. Extracting the timestamp into a column achieves the intent, location never reaches storage and capture time survives, and it is verifiable.
- Dimensions are read directly from the PNG, JPEG and WebP headers, so no native image dependency is required to store an image. Resizing to a 512 pixel thumbnail uses sharp when it happens to be installed; when it is not, no thumbnail row is written and the thumbnail endpoint falls back to the original.
- Classification routes each image to a matching analysis prompt: skin analysis, prescription extraction, document extraction and explanation, or growth chart reading.
- Blobs are stored in PostgreSQL by default, with a local disk driver available behind `STORAGE_DRIVER`.

2.9 Growth and immunisation

Percentiles and z-scores are computed locally from the shipped WHO Child Growth Standards LMS tables at day resolution, covering days 0 to 1856 for weight for age, length or height for age, and head circumference for age, for both sexes. There is never a network call for a growth number. Corrected age is used where a gestational age is recorded. Where sex is neither male nor female the female table is used and the result is flagged, rather than silently reporting a number the standard cannot support.

Immunisation merges the Nepal EPI schedule with what has actually been recorded, producing a given, due, upcoming and overdue view with a configurable overdue grace period. Vaccine name matching is deliberately forgiving, since parents and clinics write the same vaccine many different ways, and it falls back to the dose number when a label matches several entries.

2.10 Medication normalisation

Whatever a parent typed, or whatever OCR read off a handwritten slip, has to become a generic drug name the interaction model has a chance of knowing. The raw name is always preserved on the medication row. Normalisation is a hint, never a replacement.

8. Lowercase, strip the Devanagari danda, strip strength patterns such as 2mg/5ml, strip form words such as syrup, drops, tablet, forte and paediatric, then strip punctuation while keeping hyphens.
9. Match against the seeded alias table, 88 entries covering brand names common in Nepal and India.
10. Fall back to trigram similarity, the same measure `pg_trgm` uses, implemented in JavaScript so normalisation needs neither a database round trip nor the extension.

2.11 Follow-ups and reminders

When something is being watched, Copi schedules a check-in. It does not schedule one on the same turn that the parent tells it the problem has cleared up, because scheduling a how is the fever today message while closing the fever is exactly the behaviour that makes an assistant feel like it is not listening.

A reply that answers a pending check-in is linked back to it, so the follow-up moves to answered rather than sitting at sent and later being marked missed. Escalation, snoozing and cancellation are all exposed on the follow-up routes.

2.12 The job queue

Copi uses a `copi_jobs` table with `SKIP LOCKED` claiming rather than `pg-boss`. `pg-boss` depends on `LISTEN` and `NOTIFY` and installs its own schema, and this database is reached through a connection pooler where `LISTEN` does not survive. The table delivers everything needed in roughly 200 lines with no new dependency.

- At-least-once delivery, so every handler is written to be idempotent
- Exponential backoff between attempts, dead lettered as failed at exactly `maxAttempts`
- Scheduled execution through `run_at`
- Debounce and coalescing through a partial unique index on `singleton_key`. Re-enqueuing a pending key pushes its `run_at` out and merges the newest payload, which is what turns a burst of five messages into one twin rebuild
- Stale lock reclamation after five minutes, so a worker crash does not strand a job

The sixteen registered handlers:

<code>message.process</code>	<code>media.classify</code>	<code>followup.send</code>
<code>media.analyze</code>	<code>media.ocr</code>	<code>followup.escalate</code>
<code>embeddings.index</code>	<code>twin.rebuild</code>	<code>medication.remind</code>
<code>ddi.check</code>	<code>twin.translate</code>	<code>vaccination.remind</code>
<code>ddi.retry</code>	<code>push.send</code>	<code>episode.autoresolve</code>
<code>cleanup.orphans</code>		

The cost of this design is polling rather than push, so a job starts within one poll interval, two seconds by default, instead of immediately. That is irrelevant at this scale, and `POST /api/v1/dev/jobs/drain` runs the queue synchronously for demos and tests.

2.13 The language model layer

Every driver satisfies the same four method interface:

```
complete(req)          -> { text, model, provider, promptTokens, completionTokens }
completeStructured(req) -> a validated object matching req.schema
vision(req)             -> the same shape as complete, with req.images
embed(texts, dimensions) -> number[][]
```

ResilientProvider wraps a primary and an optional fallback driver and adds a 30 second timeout, two retries with jitter, a circuit breaker that opens after five consecutive failures and half opens after 30 seconds, automatic failover, structured output validation with a single repair pass, and a per call log line persisted to `copi_llm_calls`.

The repair pass matters in practice. Models wrap JSON in prose or a code fence even when told not to, so the extractor unwraps fences and falls back to the outermost balanced object. If Zod still rejects the result, the Zod error is fed back to the model verbatim, which fixes the overwhelming majority of cases in one attempt.

The fallback provider is deliberately not the fixture driver by default. Fixtures invent specifics, so falling back to them behind a real provider would mean a parent asking about feeding gets confident advice about a rash. When the primary is real, no fallback is configured unless someone opts in, and callers degrade honestly instead.

Ten prompt files are committed as Markdown under `llm/prompts` and rendered with named placeholders: `analyze_skin`, `classify_media`, `compose_followup`, `explain_document`, `extract_document`, `extract_events`, `extract_prescription`, `generate_guidance`, `rebuild_twin_summary`, `respond_briefly`, `translate_ne` and `triage_assess`.

Setting	Default
Text model	gemini-3.5-flash-lite
Vision model	gemini-3.5-flash-lite
Embedding model	gemini-embedding-001 at 768 dimensions
Timeout	30,000 ms
Retries	2
Circuit breaker	5 consecutive failures, 30 second cooldown
Context token budget	6,000
Local alternative	Ollama with qwen2.5:7b-instruct, gemma3:4b, nomic-embed-text

Flash Lite is chosen over Flash deliberately. One inbound message costs roughly eight model calls, and the free tier allowance on the larger model is far too small for that. Flash Lite handles vision and structured extraction well enough for this workload.

2.14 Language handling

Language detection is hand written rather than delegated to `franc-min`, because the case that matters is exactly the one general purpose detectors get wrong. Romanized Nepali is Latin script with no diacritics,

so franc-min classifies baby lai hijo dekhi jwaro cha as English. lib/language.js combines a Devanagari codepoint ratio with a weighted Romanized Nepali token list and returns ne, ne-Latn, mixed or en. It is verified against pure Devanagari, Romanized Nepali, code switched and pure English fixtures.

2.15 Data model

Twenty nine Sequelize models, all `copi_` prefixed, registered on the same Sequelize instance as the existing application so a single `initDatabase()` syncs both. They are grouped by domain rather than one file per model, with `models/index.js` declaring every association in one place.

Group	Models
identity	Child, Caregiver, CaregiverInvite, PushSubscription
threads	Thread, ThreadParticipant, Message, MessageAttachment
media	Media, MediaBlob
timeline	Event, Episode, EventAttachment
twin	ChildProfile, GrowthMeasurement, Embedding
medications	Medication, MedicationDose, DrugAlias, DdiCheck, DdiCache
safety	RedFlagRule, SafetyEvaluation, GuidanceSession, FollowUp
ops	AuditLog, AppSetting, LlmCall, Job

Notable relationships:

- A child has exactly one profile, the digital twin, and cascades on delete.
- An event optionally belongs to an episode; deleting an episode sets the link to null rather than removing history.
- Events form follow-up chains through `parentEventId`, so a check-in answer points back at the event it answers.
- Events and messages both attach to media through join tables, so one photograph can belong to a message and a timeline entry at once.
- A thumbnail is its own media row pointing back at the original through `parentMediaId`.
- Embeddings are stored as a JSONB float array rather than a pgvector column. Sequelize has no native vector type, so a real vector(768) column would need raw SQL outside the model layer and a hand maintained migration. One child's corpus is in the low thousands of chunks, and a linear scan over that is sub-millisecond. `semanticSearch()` is the only caller, so swapping in pgvector later is a one file change.

2.16 The HTTP API

Everything is served under `/api/v1`, sharing the analyzer's session cookie. Every request carries a request id that is propagated into job payloads, so one message's entire journey is greppable by a single value. A global rate limit of 100 requests per minute is applied at the router.

`GET /api/v1/readyz` is the probe to check first. It reports the database, the job queue and its handler count, the interaction service and its vocabulary size, the language model provider and its circuit state, the active

channels, and whether push is configured. The database is the only hard dependency; everything else degrades.

Group	Representative endpoints
Health	GET /healthz, GET /readyz, GET /
Children	GET POST /children, GET PATCH DELETE /children/:childId, GET /children/:childId/dashboard, GET /children/:childId/avatar
Caregivers	GET /children/:childId/caregivers, PATCH DELETE /children/:childId/caregivers/:userId, POST /children/:childId/invites, POST /invites/:code/redeem
Timeline	GET POST /children/:childId/events, GET /children/:childId/events/pending, GET PATCH DELETE /events/:eventId, POST /events/:eventId/confirm, POST /events/:eventId/reject
Episodes	GET /children/:childId/episodes, GET PATCH /episodes/:episodeId
Twin	GET PATCH /children/:childId/profile, POST /children/:childId/profile/rebuild, GET POST /children/:childId/growth, GET /children/:childId/growth/percentiles, GET POST /children/:childId/vaccinations
Medications	GET POST /children/:childId/medications, PATCH /medications/:medicationId, POST /medications/:medicationId/confirm, POST /medications/:medicationId/stop, POST /medications/:medicationId/doses, POST /medications/remind
Interactions	POST /children/:childId/medications/check, POST /children/:childId/prescriptions/scan, GET /children/:childId/ddi-checks, POST /ddi-checks/:checkId/acknowledge
Guidance	POST /children/:childId/guidance, GET /guidance/:guidanceId, GET /guidance/:guidanceId/context, POST /guidance/:guidanceId/feedback
Media	POST /children/:childId/media, GET DELETE /media/:mediaId, GET /media/:mediaId/thumbnail, GET /media/:mediaId/analysis, POST /media/:mediaId/ask, GET /children/:childId/documents
Threads	GET /children/:childId/threads, POST /children/:childId/threads/link, POST /children/:childId/threads/:threadId/rebind, PATCH /threads/:threadId, POST /threads/:threadId/disconnect, GET /threads/:threadId/messages, GET

	/threads/:threadId/participants, POST /threads/:threadId/participants/:participantId/claim
Follow-ups	GET /children/:childId/follow-ups, POST /follow-ups/:followUpId/snooze, POST /follow-ups/:followUpId/cancel
Safety and audit	GET /children/:childId/safety-evaluations, GET /children/:childId/search, GET /children/:childId/export
Notifications	GET /notifications/vapid-public-key, POST DELETE /notifications/subscribe, POST /notifications/test
Webhooks	POST /telegram/:secret, GET POST /whatsapp
Dev only	POST /dev/simulate/telegram, POST /dev/jobs/drain, GET /dev/jobs, POST /dev/follow-ups/:id/trigger, POST /dev/time-travel, GET /dev/messages/:id/trace

The dev routes are mounted only when `NODE_ENV` is not production. They run the identical twelve stage pipeline offline, which is what makes the whole system demonstrable and testable with no Telegram bot, no public URL and no tunnel.

2.17 Channels

Channel	Transport	Notes
telegram	Long polling by default, webhook optional	Polling works from localhost with no public URL, which is why it is the primary channel. Pinned to IPv4, because <code>api.telegram.org</code> resolves to IPv6 first and on a host without working IPv6 every call hangs in a way that looks exactly like a bad token.
whatsapp	Cloud API webhook	The raw request body is preserved through JSON parsing so the HMAC signature can be verified over the exact bytes received.
upload	Direct HTTP upload	Always active. The path used by the web client and by the offline simulator.

A chat is bound to a child by a six digit link code. The bot replies once with the code, the caregiver enters it in the app, and only then does anything from that conversation get ingested. The code is sent once per code rather than once per thread, so a chat that is disconnected and given a fresh code is told the new one.

2.18 Frontend

React 19 with Vite, React Router 7, Tailwind CSS 4 and Radix UI primitives. Vite proxies `/api` to the backend so the session cookie stays first party.

Route	Screen
/	Copi overview, the whole home screen from one dashboard request
/timeline	The full timeline, filterable and searchable
/medicines	Drafts, interactions, prescription scan, and the before I buy this check
/reports	Exports and reports
/ask	The ask box, with why did it say that
/review	Low confidence extractions waiting for a human
/twin	Full profile, growth, vaccinations and chat linking
/episodes/:episodeId	One episode and its events
/analyzer	The drug interaction analyzer, unchanged
/login, /register	Authentication, reachable only when signed out

Links written against the older /copi prefix still work. /copi/timeline redirects to /timeline rather than dropping the visitor at the root, and the redirect reuses the same route declarations so it cannot drift.

3. The drug interaction engine

3.1 Problem

When a patient takes several drugs at once, interactions can occur through shared protein targets and cause serious adverse effects. The number of possible combinations from 2,135 drugs alone is close to two million. No database covers them all, and existing tools flag only known pairs with a binary yes or no.

A Graph Neural Network trained on a biological drug-protein-protein interaction graph can reason about combinations that were never explicitly tested, because it learns from the shared biology rather than from an enumerated list.

3.2 Graph and training data

Property	Value
Nodes	21,257 (2,135 drugs and 19,122 proteins)
Edges	1,820,238
Source	DECAGON, Stanford SNAP
Training records	4.6 million polypharmacy side effect records
Labelled pairs	63,473 drug pairs
Side effect classes	1,317
Class balance	30.2 percent harmful

3.3 Architecture

DrugInteractionGNN combines a GraphSAGE encoder, a DEDICOM style diagonal bilinear decoder and an MLP harmfulness head.

11. Input projection. Drug features are projected to the hidden dimension by a linear layer; proteins are given a learned 64 dimensional embedding and projected to the same space, then the two node types are concatenated into one node matrix.
12. Layer 1, relation specific. Three separate SAGEConv operators run over the drug-drug, drug-protein and protein-protein edge sets, and their outputs are summed. This follows Decagon section 4.1: a protein-protein edge does not mean the same thing as a drug-protein edge, so they should not share weights. Batch normalisation, ReLU and dropout follow.
13. Layer 2, shared. A single SAGEConv over the full edge index projects to a 32 dimensional embedding, again with batch normalisation, ReLU and dropout.
14. Side effect decoding. For each of the 1,317 side effects there is a learned diagonal matrix D_r , and one global relation matrix R shared across them. The score for a pair is $z_i^T \text{diag}(D_r) R \text{diag}(D_r) z_j$. Factoring the shared structure into R keeps the per side effect parameter count linear in the embedding dimension rather than quadratic.
15. Harmfulness head. The 1,317 sigmoid side effect probabilities, optionally weighted by side effect frequency, feed a two layer MLP that emits a single harm logit for the pair.

3.4 Performance

Metric	Score
ROC-AUC	0.884
Recall, sensitivity	0.988
Average precision	0.747
F1	0.531

The high recall is intentional and the precision cost is accepted deliberately. In a safety tool, missing a real interaction is far worse than raising a false positive that a clinician dismisses in a second.

3.5 Scoring and severity

Raw model output is turned into clinician facing numbers in `src/predict.py`. Side effects are reported above a per pair adaptive threshold, the mean plus 1.5 standard deviations, with a global floor of 0.50, capped at the top 20 overall and the top 5 per pair.

Each side effect is assigned a severity, first from the DECAGON effect category table and then, for effects the table does not cover, from a keyword fallback over the effect name. Cardiovascular and haematopoietic effects map to CRITICAL, respiratory and renal or hepatic to HIGH, gastrointestinal and neurological to MODERATE, everything else to LOW.

The combined polypharmacy score is:

```
se_contribution = min(1.0, sum of per-effect severity weights)
                    CRITICAL 0.15, HIGH 0.08, MODERATE 0.03, LOW 0.01
raw   = max_harm * 0.65 + se_contribution * 0.35
score = clamp(round(raw * 100), 0, 100)
```

Score	Grade
80 and above	CRITICAL
65 to 79	HIGH
40 to 64	MODERATE
20 to 39	LOW
below 20	MINIMAL

3.6 The FastAPI service

Endpoint	Purpose
GET /	Health check
GET /drugs	The model's full identifier vocabulary
POST /predict	Scores every pairwise combination of the submitted drugs. At least two names are required.
POST /explain	A three sentence clinical explanation of a flagged combination, generated by Gemini with a deterministic fallback when no key is set.

The model is loaded once at application startup through a lifespan handler rather than per request. An unrecognised drug name raises HTTP 400 with a detail string that names the offending drug.

3.7 The Node orchestrator and analyzer client

The Express application is a caching orchestrator between the React client and the Python service. It provides cookie JWT authentication over a doctors table, per doctor patient records and analysis history, a drug catalogue service, and an interaction cache so a repeated combination does not re-run inference.

```
POST /api/auth/register login logout    GET /api/auth/me
GET  /api/patients    POST /api/patients
PATCH DELETE /api/patients/:id
GET  POST DELETE /api/patients/:id/history
GET  /api/drugs      POST /api/predict  GET /api/history
GET  /api/health
```

The analyzer front end scores combinations, renders an interaction network, breaks results down by side effect and organ system, exports a PDF report through jsPDF, and connects to a pharmacology chatbot pre-loaded with the detected interaction context.

4. The contract between Copi and the interaction model

Copi does not run its own interaction model. It calls the FastAPI service through the existing backend/src/services/mlService.js. Neither the Python service nor mlService.js was modified. The contract is recorded explicitly so that a change to the model is a change someone makes deliberately.

4.1 The call

Copi always sends exactly two names:

```
POST /predict { "drugs": ["ondansetron", "cetirizine"] }
```

The response is validated with Zod on arrival, so a shape change surfaces as a clear error rather than a silent undefined. Copi reads harmful, confidence, risk_level, risk_grade, polypharmacy_score, side_effects and, above all, pair_details[0].pair_harm_score.

4.2 Severity mapping

The model emits a per pair harm score in the range 0 to 1 and a coarse grade for the whole set. Copi maps that onto its own enum in a single function, mapSeverity, so the thresholds are tunable in one place after a retrain.

Condition	Copi severity
risk_grade is CRITICAL and score at or above 0.8	contraindicated
score at or above 0.8	major
score at or above 0.6	moderate
score at or above 0.4	minor
otherwise	none
non-numeric score	none, never an interaction

4.3 Vocabulary alignment

This is the part most likely to surprise a reader. The model's name vocabulary is 83 entries in the USAN convention. Copi normalises toward the INN convention used in Nepal and India. Without a bridge, every paracetamol check would return unknown drug, which is safe but means the feature never fires.

modules/ddi/vocabulary.js reads the model's own data/mappings/drug_name_to_stitch.json as the source of truth, so the two cannot drift apart, and applies four rules in order:

16. Exact match. The model already knows the name.
17. True synonyms only. paracetamol to acetaminophen, salbutamol to albuterol, frusemide to furosemide. Same molecule, two naming conventions.
18. Combination products by component. amoxicillin-clavulanate to amoxicillin, with clavulanate reported in the unchecked list and a note the parent can read.
19. Otherwise unsendable, and reported as not checkable.

prednisolone is deliberately not mapped to prednisone. They are different molecules, and mapping them would raise the hit rate by lying. Two names that resolve to the same molecule, such as paracetamol and a paracetamol combination, are reported as a double dose risk rather than as a meaningless self comparison.

4.4 An unknown drug is never rendered as safe

When a name is out of vocabulary the service returns HTTP 400 naming it. Copi treats that as a definite answer rather than a transient failure: it does not retry, and it does not read as safe. checkPair returns unknownDrug: true with copy that says the check could not run and to confirm with a pharmacist. The severity field is set to none only to satisfy the enum, and every consumer branches on unknownDrug rather than on severity.

The same holds when the service is unreachable. The check is stored as unavailable, the parent is told plainly that the automatic check did not run, and a retry is queued. Silence would read as a clean result.

4.5 Resilience

- 15 second timeout, two retries with backoff
- Circuit breaker opening after five consecutive failures and half opening after 30 seconds
- Results cached in copi_ddi_cache, keyed on the sorted normalised pair plus DDI_MODEL_VERSION, so bumping that version invalidates every entry at once

4.6 Triggers and consequences

A check runs when:

- A medication moves from draft to active. It is checked against every other active medication.
- POST /children/:childId/medications/check is called with an arbitrary name against the active list, writing nothing.
- POST /children/:childId/prescriptions/scan produces a preview for each extracted draft.

Severity	Stored	In thread	Push	Other
none, minor	yes	no	no	Visible in the medication detail
moderate	yes	yes	no	None
major, contraindicated	yes	yes	yes	Fires DDI_MAJOR, returns the medication to draft, opens an acknowledgement follow-up
unavailable	as unavailable	yes	no	Retry queued

This is the sharpest behaviour in the product. A major interaction does not merely warn. It refuses to let the medicine go active, posts into the family chat, and pushes to the owner and editor caregivers. The thread copy stays calm and names both drugs, and every one of these messages is checked against the tone linter in the test suite.

4.7 Swapping the model

A replacement model needs three things:

20. POST /predict accepting {drugs: [a, b]} and returning pair_details[0] with a pair_harm_score in the range 0 to 1.
21. A 400 response whose detail names an unrecognised drug.
22. A data/mappings/drug_name_to_stitch.json listing the names it accepts.

Then bump DDI_MODEL_VERSION to invalidate the cache and revisit the thresholds in mapSeverity. backend/tests/ddi.test.js covers the mapping, the vocabulary rules and the parent facing copy without needing the service running.

5. Engineering decisions

Every significant judgment call, what was decided, why, and what it costs. Recorded so that a future reader can tell a considered trade-off from an oversight.

Decision	Rationale and cost
Copi lives inside the existing backend rather than a new monorepo	The repository already contains a working product with its own auth, patients, history and ML proxy, and the requirement was to build in the existing backend and frontend without touching the interaction code. A parallel TypeScript toolchain in the same folder, or a rewrite of the existing routes, would have violated that. Cost: no compile time type safety, mitigated by validating every boundary at runtime with Zod, meaning HTTP bodies, model output and the interaction service response.
Sequelize rather than Drizzle	Follows from the above. The existing models are Sequelize on a shared connection, and two ORMs against one database would mean two migration histories. Cost: no first class pgvector support.
A PostgreSQL backed job table rather than pg-boss	pg-boss needs LISTEN and NOTIFY, which do not survive a connection pooler. The table gives retries, backoff, dead lettering, scheduling and debounce in about 200 lines with no new dependency. Cost: polling latency of one interval.
Embeddings as JSONB with cosine scored in Node	A real vector(768) column would need raw SQL outside the model layer and a hand maintained migration. A linear scan over one child's few thousand chunks is sub-millisecond. The property that matters, filtering child_id in the database WHERE clause, is preserved regardless of storage format. Cost: O(n) per child, isolated to one function.
The existing account system is Copi's identity layer	Adding a second identity system would mean two account tables and an ambiguous answer to who is signed in. copi_caregivers.user_id is a foreign key to doctors.id. Cost: no Google sign-in. The RBAC ladder Copi adds on top is fully implemented and independent of how the user authenticated.
Vocabulary bridging between INN and USAN naming	Not anticipated in the original design. Without it every paracetamol check would return unknown drug, safe but useless. Bridged for true synonyms only, reading the model's own mapping file so the two cannot drift.
EXIF timestamp extracted, then the whole block dropped	Surgically deleting only the GPS IFD means rewriting the TIFF tree and fixing every offset. Extracting the timestamp into a first class column achieves the intent and is verifiable. Verified against a JPEG carrying both a GPS IFD and DateTimeOriginal.

Image resizing optional, dimension probing not	sharp is a native build that turns npm install into a compile step. Dimensions are read from the image headers directly; resizing uses sharp only when it happens to be installed. Nothing downstream breaks either way.
PDF export hand written	About 60 lines emit a valid PDF 1.4 rather than adding a library for one endpoint, and the output is verified by parsing it back and extracting the text. Cost: Latin-1 and the base-14 fonts only, so Devanagari cannot be rendered. The JSON export carries the full bilingual content.
Language detection hand written rather than franc-min	The case that matters, Romanized Nepali, is exactly the one general purpose detectors get wrong, because it is Latin script with no diacritics.
Models grouped by domain, not one file per model	29 models in 29 files would obscure the shape of the data. Grouped into eight domain files with all associations declared in one place.
DB_SYNC selects the schema reconciliation mode	29 new tables reconciled with alter on every start made restarts slow against hosted PostgreSQL, a regression in the existing development loop. DB_SYNC now selects alter, sync or none.

6. Testing

Suite	Command	Covers
Unit	npm test	82 tests with no database, no network and no API keys: safety rules and age boundaries, tone linting, drug interaction mapping and vocabulary, medication normalisation, event extraction, WHO growth maths, growth cards, response mode selection and time handling.
Access control	npm run test:rbac	69 integration probes across the owner, editor and viewer matrix. Needs the server running.
Model	drug_interaction_gnn.ipynb	The data pipeline, training loop and held-out evaluation for the GNN.

The test suite runs entirely offline. With no GEMINI_API_KEY set, a committed fixture provider answers every model call, so the whole system boots, seeds, serves and passes its tests with zero credentials.

6.1 Verified behaviours

- Job queue: success, coalescing three enqueues into one row, deferral of not yet due jobs, retry then recover, and dead lettering at exactly maxAttempts.
- Safety: one case per rule, plus the neonatal fever age boundary tested in both directions at 88, 90 and 400 days.
- Tone: the fallback template is itself run through the linter, so the fallback can never violate the spec it exists to protect.
- Interactions: severity mapping, the vocabulary rules including the deliberate prednisolone omission, and the exact parent facing copy, all without the Python service running.
- EXIF: a JPEG carrying both a GPS IFD and DateTimeOriginal. The timestamp is read, Exif no longer appears in the output, and the result still decodes as a valid JPEG.
- PDF export: the output is parsed back and the text extracted.
- Language: pure Devanagari, Romanized Nepali, code switched and pure English fixtures.

7. Running the system

7.1 Requirements

Component	Version	Notes
Node	22.22.1	20 or later is fine
npm	9.2.0	
Python	3.10 in a conda environment	Needs torch, torch_geometric and fastapi
PostgreSQL	16	Any reachable instance, configured in backend/.env

The Python interpreter matters. `torch_geometric` is not in a base conda environment, so running `main.py` from a plain shell fails with `ModuleNotFoundError`. Activate the dedicated environment first, or call its interpreter by full path.

7.2 Three terminals

1. the interaction model

```
conda activate ddi && python3 main.py
```

2. the API, serving both products

```
cd backend && npm install && npm run seed:ddi && npm start
```

3. the UI

```
cd frontend && npm install && npm run dev
```

Then open `http://localhost:5173` and sign in as a seeded caregiver, either the owner or the editor account.

No API keys are required. With no `GEMINI_API_KEY` set, the committed fixture provider answers every model call.

7.3 Configuration

Variable	Default	Effect
<code>DATABASE_URL</code> or <code>PGHOST</code> , <code>PGUSER</code> and so on	none	Where PostgreSQL is
<code>DB_SSL</code>	true	Set false for a plain local PostgreSQL
<code>DB_SYNC</code>	alter	Schema reconciliation at boot. Set none once the schema exists for a near instant start
<code>COPI_ENABLED</code>	true	false turns Copi off entirely, leaving the analyzer untouched
<code>GEMINI_API_KEY</code>	empty	Empty means the fixture provider answers every call

TELEGRAM_BOT_TOKEN	empty	Empty means no live chat; use the simulate route
VAPID_PUBLIC_KEY and VAPID_PRIVATE_KEY	empty	Empty means push is skipped
COPI_DEFAULT_LOCALE	ne	Default language for generated content
COPI_TIMEZONE	Asia/Kathmandu	Scheduling timezone
EXTRACTION_AUTOCONFIRM_THRESHOLD	0.75	Below this confidence an extracted event goes to the review queue instead of silently entering the twin
EPISODE_AUTORESOLVE_DAYS	14	Quiet period after which an open episode closes itself
EMBEDDING_TOP_K	8	Chunks retrieved per query
EMBEDDING_MAX_DISTANCE	0.55	Cosine distance ceiling for retrieval
MEDIA_MAX_BYTES	26214400	25 MB attachment ceiling

7.4 Useful commands

```

npm start      serve both products
npm run dev    the same, with node --watch
npm run seed   rebuild the demo data
npm run seed:ddi the same, and run a live interaction check
npm test       the unit suite
npm run test:rbac the access control probes, needs the server running
npm run vapid  generate a VAPID keypair for web push

```

7.5 The demo seed

The seed is idempotent and can be re-run at any time to reset the demo. It creates two users, one fourteen month old child whose date of birth is computed from today so the age never goes stale, 78 events across 60 days, three synthetic images generated at seed time, a resolved fever episode, an open belly rash episode, one low confidence event waiting in the review queue, a bilingual chat transcript, and the cetirizine and ondansetron pair that produces a live major interaction.

No real medical images are in the repository. The rash photograph, the prescription and the lab report are all drawn programmatically at seed time.

7.6 Diagnostics

Symptom	Cause and fix
Boot is slow	29 tables reconciled with DB_SYNC=alter on every start. Set DB_SYNC=none once the schema exists, and use alter again after changing a model.

readyz reports the interaction service down	main.py is not running, or was started from an interpreter without torch_geometric. Copi degrades either way: checks are recorded as unavailable, the parent is told plainly, and a retry is queued. It never implies safety.
The answers look canned	That is the fixture provider, which runs with no GEMINI_API_KEY. Set the key for real answers.
Nothing appears in a chat	Expected without TELEGRAM_BOT_TOKEN. readyz will list only the upload channel. Use POST /dev/simulate/telegram, which runs the identical pipeline.
A job looks stuck	The queue polls every two seconds. POST /dev/jobs/drain runs it synchronously; GET /dev/jobs shows the last 50 jobs and why any failed.
An unexpected 403	Every child scoped route requires an active caregivers row. An editor can read and write events, media and medicines but cannot manage caregivers or delete the child.

8. Design principles

Five principles run through both products and explain most of the choices above.

23. Safety is code, not a prompt. Anything that must be a guarantee is deterministic, age gated and tested. The language model may escalate a concern but may never reduce one, and its own tier suggestion is recorded alongside the rule engine's for audit.
24. Silence never reads as safety. An unknown drug, an unreachable model, a failed check: each of these is reported explicitly in the parent's own words, with a retry queued. The system says what it could not do.
25. Degrade honestly. The database is the only hard dependency. The interaction service, the language model, the chat channels and push notifications can all be absent, and the product keeps working while telling the truth about what is missing.
26. Every answer is explainable. The context, the retrieved passages with their distances, the token budget and the reconciliation between rule and model are all exposed on a route, not buried in a log.
27. Additive by construction. Copi did not modify a single existing route, service, model or component beyond mounting itself, and one environment variable removes it entirely.